Fine-tuning

Head mới trước, mở encoder có kiểm soát

Khi embedding chưa đủ

Ảnh bảng viết tay của câu lạc bộ khác miền dữ liệu nguồn.

Head nhỏ có thể chưa đọc được nhãn tốt.

CÂU HỎI

Mở phần nào của encoder, bước dài bao nhiêu, giữ checkpoint nào?

Mục tiêu buổi này

- Thay head rồi đóng băng hoặc mở phần cần học.
- Phân biệt grad mode, eval mode và optimizer.
- Đặt learning rate theo từng nhóm tham số.
- Kiểm buffer BatchNorm và chọn checkpoint bằng validation.

01

Chuyển kiến thức

Encoder và head

$$\mathbf{z} = f_\theta(\mathbf{x}), \qquad \boldsymbol{\ell} = g_\phi(\mathbf{z})$$

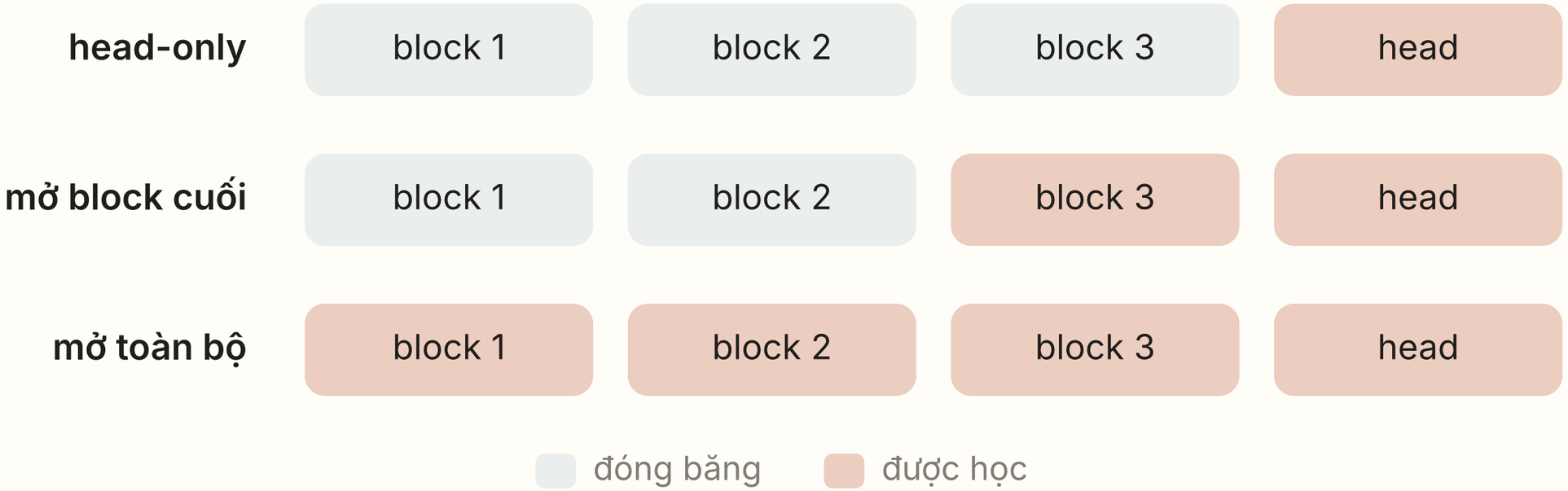
θ là encoder, ϕ là head.

Head-only: chỉ học ϕ.

Fine-tuning: học thêm phần đã chọn của θ.

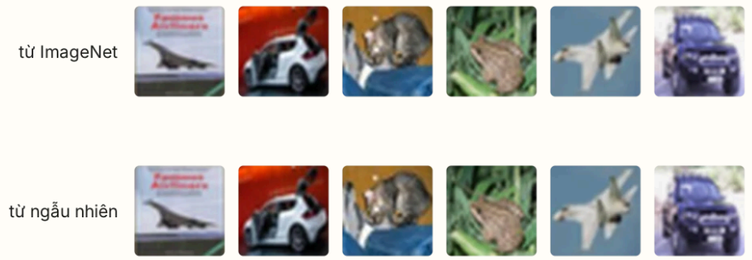
Thứ tự thử: head → block cuối → toàn bộ.

Mở có chủ đích



Ba mức mở tham số, thử theo thứ tự: chỉ head, thêm block cuối, rồi toàn bộ encoder.

Bắt đầu từ ImageNet hay từ ngẫu nhiên



bước 0: cả hai head mới đều ngẫu nhiên



bước 10: accuracy validation (400 ảnh) từ ImageNet 0.80, từ ngẫu nhiên 0.54



bước 50: accuracy validation (400 ảnh) từ ImageNet 0.86, từ ngẫu nhiên 0.59



bước 150: accuracy validation (400 ảnh) từ ImageNet 0.88, từ ngẫu nhiên 0.68

ResNet-18 học bốn lớp CIFAR-10 từ 800 ảnh, cùng số bước: bắt đầu từ trọng số ImageNet đoán đúng sớm hơn hẳn bắt đầu từ ngẫu nhiên.

HỎI NHANH

Nhiệm vụ cũ có 1000 lớp, bài mới có 3 lớp. Nạp checkpoint xong, phần nào chắc chắn phải thay hoặc kiểm trước khi train?

A Batch size, vì đầu ra 3 lớp cần batch chia hết cho 3

B Head cuối, và bảng đổi ba nhãn mới sang chỉ số 0, 1, 2

C Seed, vì head cũ được khởi tạo bằng một seed khác

D Learning rate của encoder, vì 1000 lớp cần bước nhỏ hơn

HỎI NHANH

Dữ liệu đích ít và khá giống miền nguồn, head-only đã có điểm validation. Ngân sách chỉ đủ thử thêm một cấu hình. Nên thử gì?

A Mở toàn bộ mạng, với cùng learning rate của head

B Khởi tạo lại encoder rồi học từ đầu trên dữ liệu đích

C Mở block đầu gần input, vì đặc trưng nền quan trọng nhất

D Mở thêm block cuối, giữ các block đầu đóng băng

Mô hình nguồn tự pretrain

Nguồn: chữ số **0–7**.

Đích: **8 và 9**, hoàn toàn bị loại khỏi học nguồn.

Encoder `64 → 32 → ReLU`.

KẾT QUẢ

```
source train/valid: 1082 361
source valid accuracy: 0.9806
source gap target: True
```

Đóng vai checkpoint: đã học 0–7, chưa thấy 8, 9.

HỎI NHANH

Một đội pretrain encoder trên toàn bộ 1797 ảnh chữ số từ 0 đến 9 cùng nhãn của chúng, rồi báo accuracy của nhiệm vụ phân biệt 8 với 9 trên validation lấy từ chính bộ ảnh ấy. Con số này đo gì?

A Đúng khả năng chuyển học, vì head đích vẫn là head mới

B Đúng khả năng chuyển học, vì phần nguồn không dùng head đích

C Một con số quá lạc quan, vì encoder đã học cả ảnh validation

D Một con số quá bi quan, vì nhãn 0 đến 7 làm nhiễu encoder

02

Freeze và learning rate

Head mới, encoder giữ nguyên

```
for parameter in head_only[0].parameters():
    parameter.requires_grad_(False)
optimizer = torch.optim.SGD(
    head_only[1].parameters(), lr=0.05)
```

48 ảnh train đích, 306 ảnh validation.

Head 32 → 2 có **66 tham số**.

Đổi nhãn 8 thành 0, nhãn 9 thành 1.

Kiểm đúng phần nào thay đổi

KẾT QUẢ

```
encoder grad None: True
head co gradient: True
encoder khong doi: True
```

Gradient đúng chưa đủ: so cả tensor trước/sau optimizer.

Loss giảm không cho biết encoder có bị thay đổi hay không.

HỎI NHANH

Một tham số có `requires_grad=False` nhưng vẫn nằm trong forward. Khi dự đoán, nó đóng vai trò gì?

A Vẫn tính forward như cũ, chỉ không được học

B Bị bỏ qua, nên output của tầng ấy bằng 0

C Bị thay bằng giá trị trung bình của nó

D Bị tách khỏi mạng cho tới khi gọi lại `train()`

HỎI NHANH

Head hai lớp được học với nhãn `digit - 8`. Validation accuracy đạt 0.94, nhưng file nộp ghi thẳng `argmax` của logits và được chấm 0 điểm. Lỗi khả dĩ nhất là gì?

A Head hai lớp không đủ sức cho dữ liệu test

B Validation bị rò, nên 0.94 là con số ảo

C File nộp ghi chỉ số 0, 1 thay vì 8, 9

D Encoder ở chế độ `eval()` khi dự đoán test

Một bước cập nhật tính tay

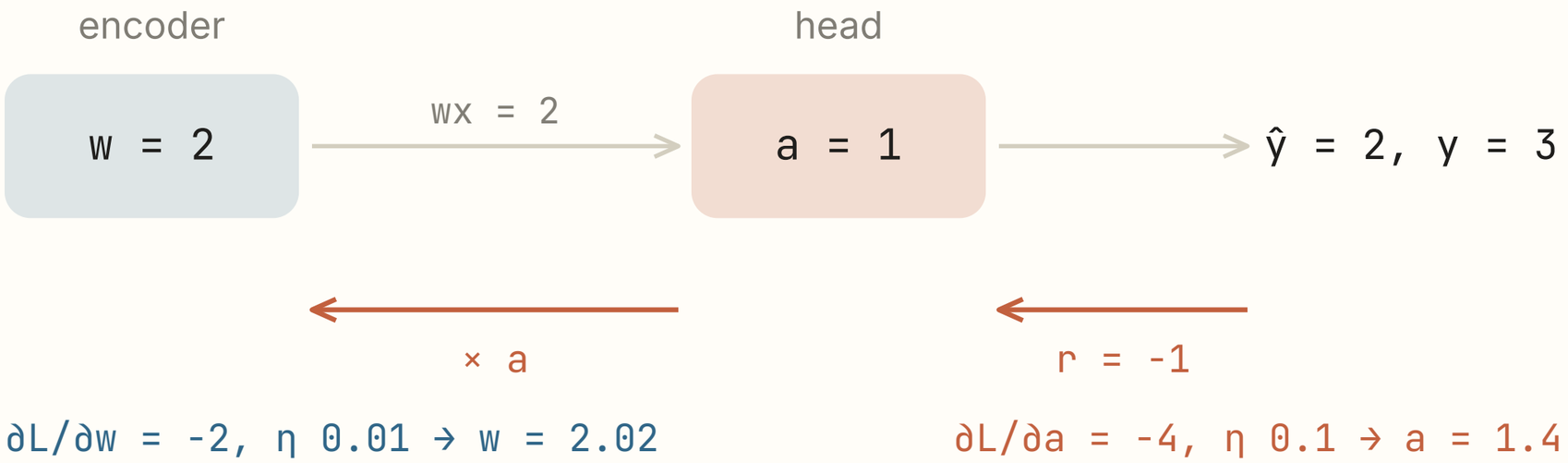
$$\hat{y} = a(wx), \quad L = (\hat{y} - y)^2$$

$$x = 1,\ w = 2,\ a = 1,\ y = 3.$$

Gradient: $\partial L/\partial a = -4,\ \partial L/\partial w = -2.$

$$\eta_a = 0.1,\ \eta_w = 0.01 \text{ cho } a = 1.4,\ w = 2.02.$$

Gradient về encoder nhân qua head



Ví dụ tính tay ŷ = a(wx): gradient về encoder phải nhân qua a của head; với learning rate 0.1 cho head và 0.01 cho encoder, a lên 1.4 còn w chỉ nhích tới 2.02.

Gradient của encoder đi qua head

$$\frac{\partial L}{\partial w} = 2(\hat{y} - y) \cdot a \cdot x$$

Head mới còn ngẫu nhiên: encoder bị kéo để bù cho head.

Bước lớn dễ làm hỏng đặc trưng đã học: **quên thảm khốc**.

Cách giữ: rate encoder nhỏ, hoặc head-only trước rồi mới mở.

BERT fine-tune với learning rate từ $2 \cdot 10^{-5}$ đến $5 \cdot 10^{-5}$.

HỎI NHANH

Vẫn ví dụ ấy, với $x = 1$, $w = 2$, $y = 3$, nhưng head mới được khởi tạo $a = -1$. Gradient theo w bằng bao nhiêu, và gradient descent đẩy w theo hướng nào?

A -2, nên w tăng, như ví dụ có $a = 1$

B -10, nên w tăng để $\hat{y}$ lớn lên

C 0, nên w đứng yên chờ head học xong

D 10, nên w giảm

Hai nhóm learning rate

```
groups = [{
    "params": transferred[1].parameters(),
    "lr": 0.05,
}]
if open_encoder:
    groups.append({
        "params": transferred[0].parameters(),
        "lr": 0.003,
    })
optimizer = torch.optim.Adam(groups)
```

Head mới dùng bước lớn hơn; encoder dùng bước nhỏ hơn.

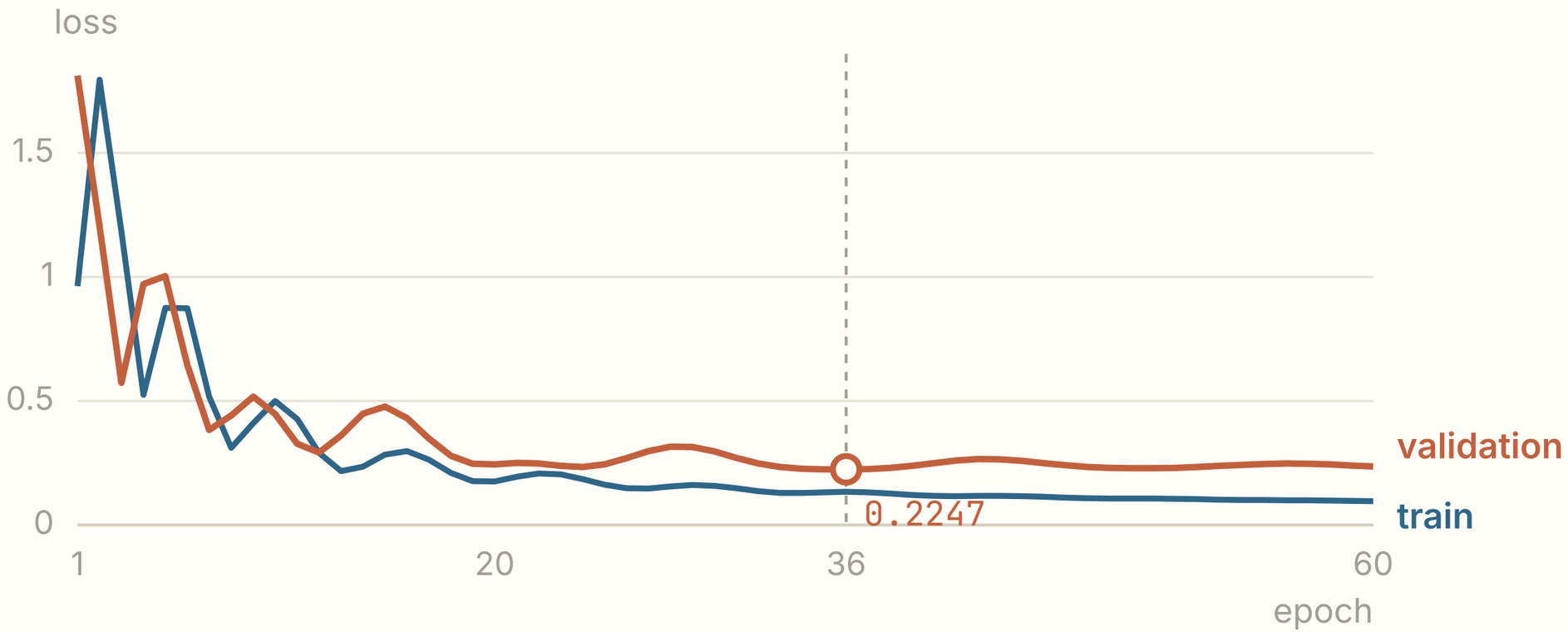
Tỉ lệ khoảng 17 lần là một lựa chọn, không phải hằng số.

Mở block nhưng quên thêm vào optimizer thì block không được cập nhật.

03

Validation và trạng thái tầng

Chọn checkpoint từ validation



Head-only (số từ code bài): train loss giảm tới epoch 60, validation thấp nhất 0.2247 ở epoch 36 rồi nhích lên, nên checkpoint giữ epoch 36.

So sánh cùng mốc và cùng split

Hai mode dùng cùng encoder nguồn, head mới và split đích.

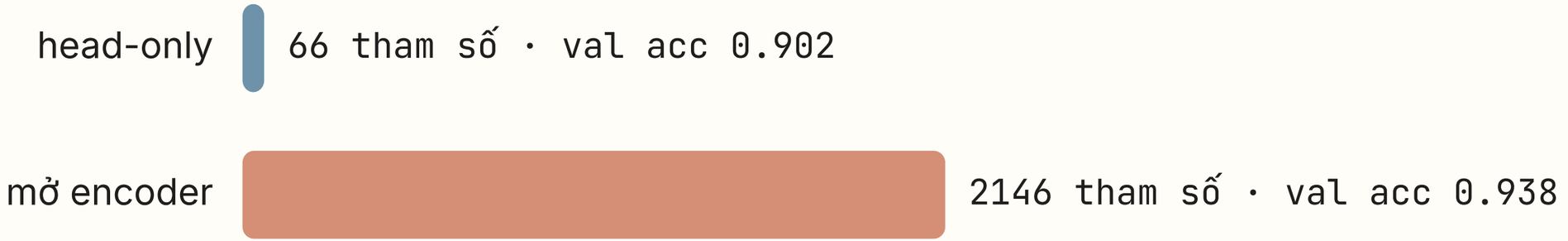
Mode	Tham số học	Epoch chọn	Val accuracy
Head-only	66	36	0.9020
Mở encoder	2146	60	0.9379

Chọn bằng **validation loss**:

- Head-only: 0.2247 (epoch 60: 0.2366).
- Mở encoder: 0.1570, đúng epoch cuối.

Tốt nhất ở epoch cuối: có thể chưa đủ epoch.

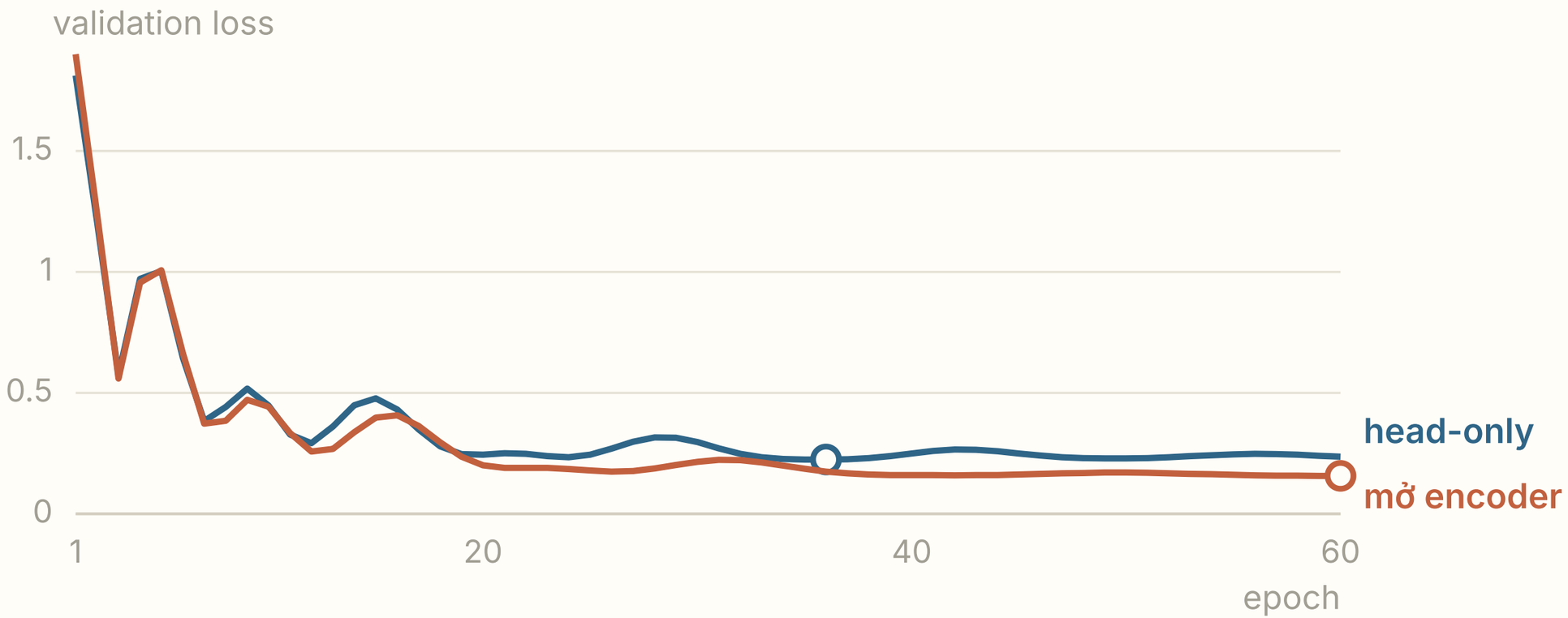
Tham số học và accuracy



đúng thêm 11 trên 306 ảnh validation; mỗi tham số mở kèm gradient và hai moment Adam

Số tham số được học và accuracy validation ở checkpoint (số từ code bài): mở encoder tăng từ 66 lên 2146 tham số, accuracy từ 0.902 lên 0.938.

Validation loss của hai cách



Validation loss của hai cách (số từ code bài, chấm tròn là checkpoint): mở encoder với learning rate nhỏ xuống 0.157 và còn đang giảm ở epoch 60; head-only dừng ở 0.2247.

HỎI NHANH

Một lần fine-tune 20 epoch có checkpoint tốt nhất ở epoch 20, và validation loss vẫn đang giảm. Hiểu kết quả này thế nào?

A Có thể chưa đủ epoch, nên chạy thêm rồi xem tiếp

B Đã hội tụ đúng ở epoch 20, nên dừng tại đó

C Đã overfit, vì điểm tốt nhất rơi vào epoch cuối

D Chọn lại theo train loss, vì validation chưa ổn

BatchNorm còn có buffer

- Weight/bias: tham số affine.
- Running mean/variance: buffer.
- Train mode có thể cập nhật buffer ở forward.

Batch mean `(3,6)`, mean cũ `(0,0)`, momentum 0.5:

$$\text{mean mới} = (1.5, 3.0)$$

Không cần gradient để có thay đổi này.

Thực nghiệm buffer

KẾT QUẢ

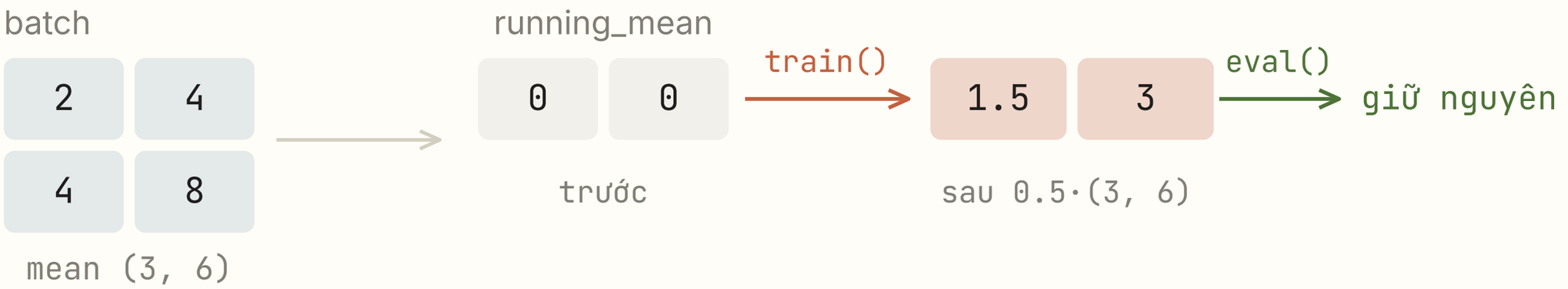
```
running mean sau train: [1.5, 3.0]
eval giu buffer: True
khop tinh tay: True
```

Sau `model.train()`:

Nếu encoder cần đứng yên, gọi `encoder.eval()`.

Freeze tham số và giữ buffer là hai quyết định.

Buffer đổi dù đã đóng băng



mọi tham số đã requires_grad=False, đang trong no_grad(), mà buffer vẫn đổi

BatchNorm đã đóng băng hết tham số vẫn cập nhật running_mean khi chạy ở train(): từ (0, 0) thành (1.5, 3); chỉ eval() giữ buffer đứng yên.

HỎI NHANH

Đóng băng weight và bias của BatchNorm, rồi cho một batch mới đi qua ở chế độ `train()`, bên trong `no_grad()`. Running mean có đổi không?

A Không, vì mọi tham số đều có `requires_grad=False`

B Không, vì `no_grad()` chặn mọi thay đổi trong forward

C Có, vì buffer tự cập nhật ngay trong forward

D Có, nhưng chỉ khi gọi `optimizer.step()` sau đó

HỎI NHANH

Encoder có BatchNorm cần giữ nguyên. Đoạn sau có giữ được running mean của encoder không?

```
encoder.eval()        # một lần
for epoch in range(10):
    model.train()
    ...
```

A Có, vì `encoder.eval()` đã được gọi từ trước

B Không, vì `model.train()` lan xuống cả encoder

C Có, nếu tham số encoder đã `requires_grad=False`

D Không, vì optimizer cập nhật buffer ở mỗi `step()`

Khi đi thi

- Baseline head-only trước, có submission hợp lệ.
- Chạy thử một batch: `param_groups`, gradient, bộ nhớ.
- Phần pretrained dùng learning rate nhỏ; BERT: $2 \cdot 10^{-5}$ đến $5 \cdot 10^{-5}$.
- Đúng processor của checkpoint.
- Đổi chỉ số về tên lớp trước khi nộp.

04

Luyện đề

LUYỆN ĐỀ 1

Accuracy chỉ khoảng 10% với 10 lớp, `fc.weight` y như lúc khởi tạo. Lỗi ở đâu?

```
for p in model.parameters():
    p.requires_grad = False
opt = torch.optim.Adam(model.parameters())
model.fc = nn.Linear(2048, 10)
```

A Optimizer tạo trước khi thay `fc`

B Backbone chặn gradient tới `fc`

C Learning rate quá nhỏ cho head mới

D Optimizer bỏ qua cả nhóm đóng băng

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 15

LUYỆN ĐỀ 2

Ta muốn giữ cố định encoder khi học head, đồng thời tiết kiệm bộ nhớ và thời gian của backward. Cách nào đạt được cả hai?

A Đặt learning rate 0 cho nhóm tham số của encoder

B Gọi `requires_grad_(False)` cho các tham số của encoder

C Gọi `encoder.eval()` trước mỗi epoch train

D Gọi `model.freeze()` một lần trước vòng lặp train

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 132

LUYỆN ĐỀ 3

BERT, AdamW với learning rate 10^{-3} cho cả mô hình. Validation loss thấp nhất sau epoch 1 rồi tăng mạnh, còn train loss vẫn giảm. Vì sao?

A Batch 32 quá nhỏ, cần gradient accumulation

B `max_length=128` cắt mất chữ

C Thiếu warmup, thêm warmup là đủ

D Learning rate quá lớn đối với trọng số pretrained

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 21

Điều cần giữ lại

- Fine-tuning học phần cần thay đổi cho nhiệm vụ mới.
- Freeze, eval, autograd và optimizer làm bốn việc riêng.
- Validation quyết định checkpoint; nhiều tham số hơn chưa chắc tốt hơn.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi learning rate encoder từ 0.003 lên 0.05, dự đoán trước validation loss rồi kiểm.

Có asset local: chạy recipe DistilBERT mode `head` và `full`, so hai `RESULT.json`.